Dimensionality reduction

Tomas Kysela

3 Dec 2024

Introduction

The goal of this tutorial is to get familiar with some basic methods for dimensionality reduction, complete you own implementation of the **Isomap algorithm** (in cooperation with *Multidimensional scaling*), experiment with its parameters and compare with other techniques of dimensionality reduction (**PCA**, **t-SNE**).

Background

The data you will be working with are vector representations of words in a latent (unknown) high-dimensional space. This representation of words, also know as word embedding, differs from standard bag-of-words (BoW, TFIDF, etc.) representations in that the meaning of the words is distributed across all the dimensions. Generally speaking, the word embedding algorithms seek to learn a mapping projecting the original BoW representation (simple word index into a given vocabulary) into a lower-dimensional (but still too high for our cause) continuous vector-space, based on their distributional properties observed in some raw text corpus. This distributional semantics approach to word representations is based on the basic idea that linguistic items with similar distributions typically have similar meanings, i.e. words that often appear in a similar context (words that surround them) tend to have similar (vector) representations.

Speciffically, the data you are presented with are vector representations coming from the most popular algorithm for word embedding known as word2vec[1] by Tomas Mikolov (VUT-Brno alumni). *Word2vec* is a (shallow) neural model learning the projection of BoW word representations into a latent space by the means of gradient descend. Your task is to further reduce the dimensionality of the word representations to get a visual insight into what has been learned.

Data

You are given 300-dimensional word2vec vector embeddings in the file *data.csv* with corresponding word labels in *labels.txt* for each line. Each of these words comes from one of 10 selected classes of synonyms, which can be recognized (and depicted) w.r.t. labels denoted in the file *colors.csv*

Tasks

1. **Load the dataset of 165 words**, each represented as a 300-dimensional vector. Each word is assigned to one of 10 clusters.

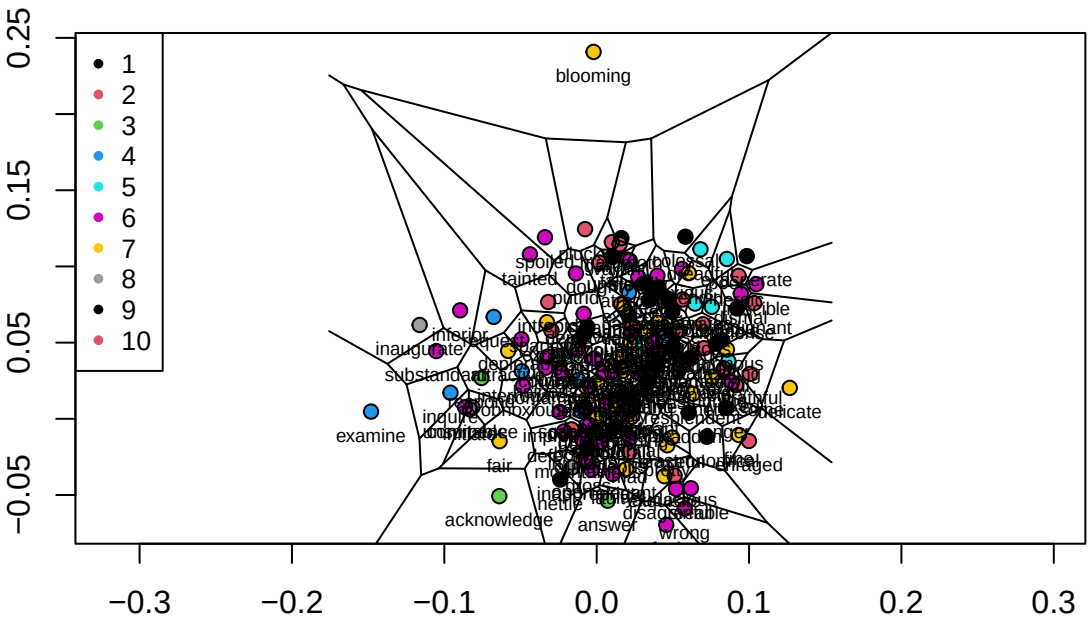
```r
#Load the dataset of 165 words
mydata <- read.csv('data.csv', header = FALSE)
mylabels <- read.csv('labels.txt', header = FALSE)
mycolors <- read.csv('colors.csv', header = FALSE)
```

The data is in the matrix `mydata`, cluster assignment in `mycolors` and the actual words (useful for visualization) in `mylabels`. You can plot the data by using only the first 2 dimensions.

[1] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. Distributed representations of words and phrases and their compositionality. In *Advances in neural information processing systems*, pages 3111-3119, 2013.

```r
PlotPoints(mydata[,c(1,2)], mylabels, mycolors)
```

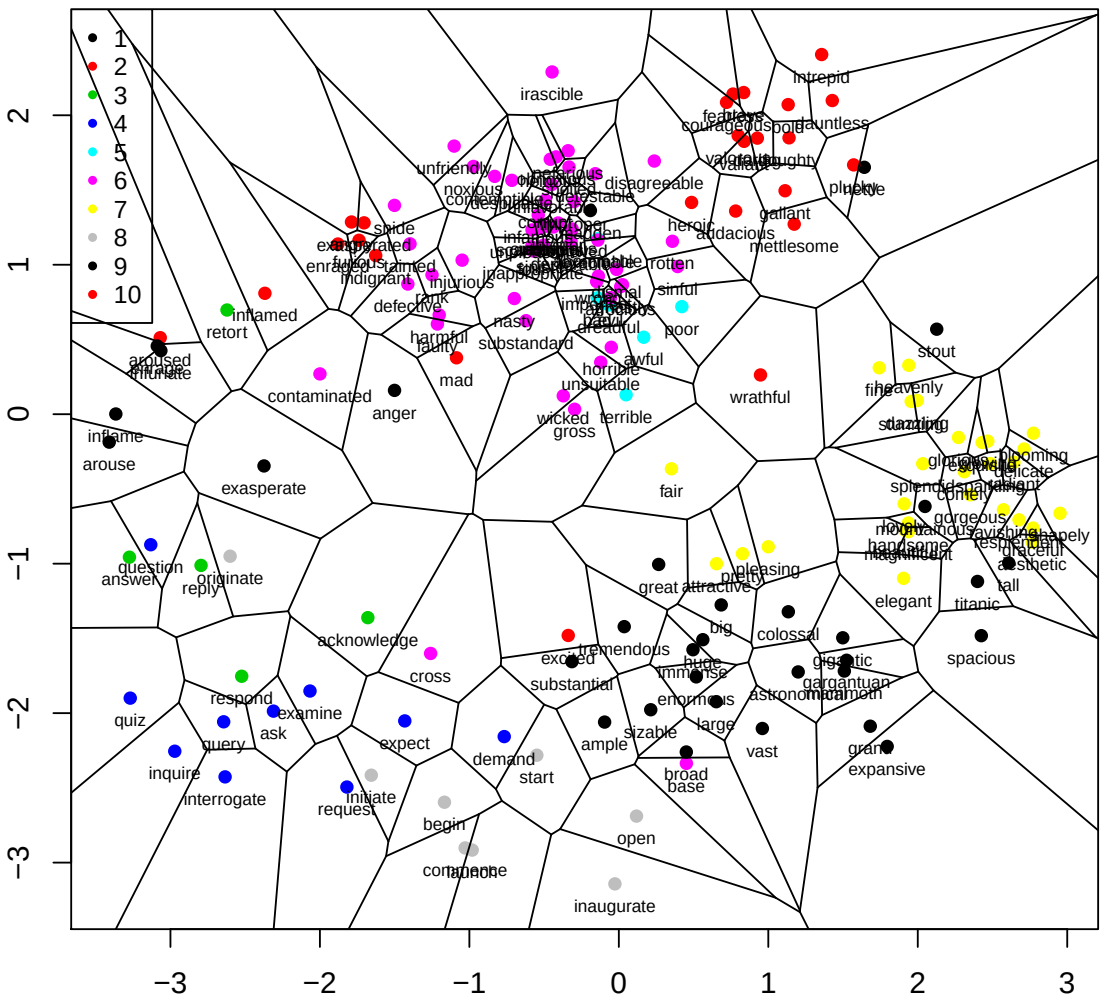
```
## deldir 2.0-4      Nickname: "Idol Comparison"

##
##      The syntax of deldir() has changed since version
##      0.0-10.  Read the help!!!.

## Warning in (function (x0, y0, x1 = x0, y1 = y0, col = par("fg"), lty =
## par("lty"), : "wpoints" is not a graphical parameter

## Warning in plot.xy(xy.coords(x, y), type = type, ...): "wpoints" is not a
## graphical parameter
```



2. **Implement ISO-MAP dimensionality reduction procedure**.

- Use k-NN method to construct the neighborhood graph (sparse matrix).
  - For simplicity, you can use `get.knn` method available in `FNN` package.
- Compute shortest-paths (geodesic) matrix using your favourite algorithm.
  - Tip: Floyd-Warshall algorithm can be implemented easily here.
- Project the geodesic distance matrix into 2D space with (Classical) Multidimensional Scaling (`cmdscale` functions in R).
- Challenge: you may simply use PCA to do the same, but be careful to account for a proper normalization (centering) of the geodesic (kernel) matrix (see Kernel PCA for details).

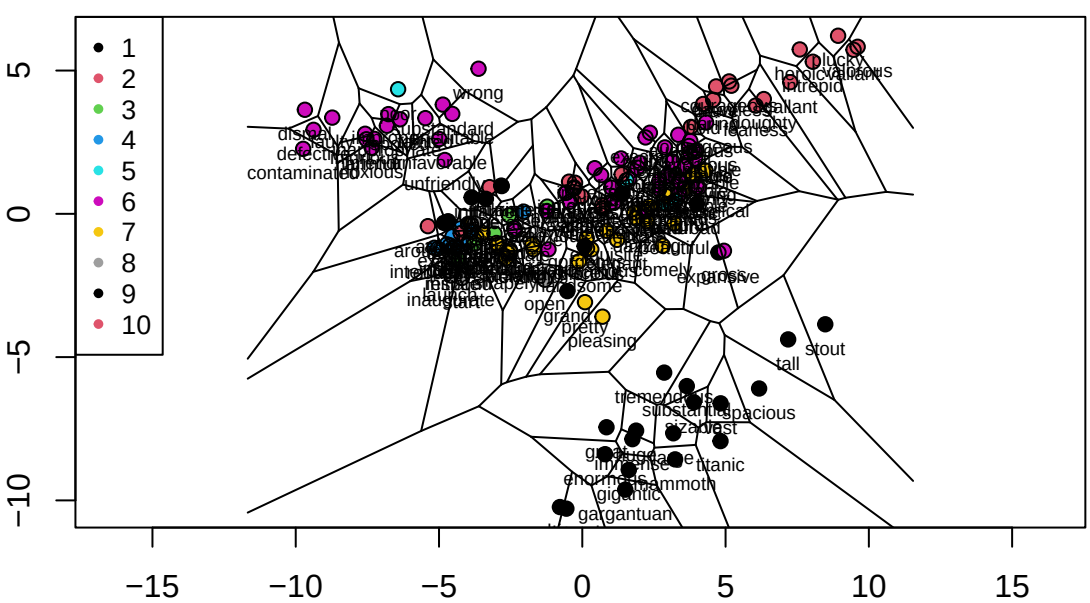
An expected result (for $k = 5$) should look similar (not necessarily exactly the same) to following




```
#ADD YOUR CODE HERE!
#REMOVE eval=FALSE arguments

#1.Determine the neighbors of each point (k-NN)
library(FNN, quietly = TRUE)
neigbours <- get.knn(mydata, k = 5)

#2.Construct a neighborhood graph.
#Each point is connected to other if it is a K nearest neighbor.
#Edge length equal to Euclidean distance.
neigbours.adjMat <- matrix(Inf, nrow(neigbours$nn.index), nrow(neigbours$nn.index))
for (i in 1:nrow(neigbours$nn.index)) {
  for (j in 1:5) {
    dest <- neigbours$nn.index[i,j]
```

```r
    dist <- neigbours$nn.dist[i,j]
    neigbours.adjMat[i, dest] <- dist
  }
}
#3.Compute shortest path between two nodes. (Floyd-Warshall algorithm)

neigbours.dist <- neigbours.adjMat

for (node.inter in 1:nrow(neigbours.adjMat)) {
  for (node.source in 1:nrow(neigbours.adjMat)) {
    for (node.dest in 1:nrow(neigbours.adjMat)) {
      neigbours.dist[node.source, node.dest] <- min(c(
        neigbours.dist[node.source, node.dest],
        neigbours.dist[node.source, node.inter] +
          neigbours.dist[node.inter, node.dest]
      ))
    }
  }
}

neigbours.dist[is.infinite(neigbours.dist)] <- 0
#4.(classical) multidimensional scaling
fit <- cmdscale(neigbours.dist,eig=TRUE, k=2)
PlotPoints(fit$points, mylabels, mycolors)
```

```
## Warning in (function (x0, y0, x1 = x0, y1 = y0, col = par("fg"), lty =
## par("lty"), : "wpoints" is not a graphical parameter

## Warning in plot.xy(xy.coords(x, y), type = type, ...): "wpoints" is not a
## graphical parameter
```



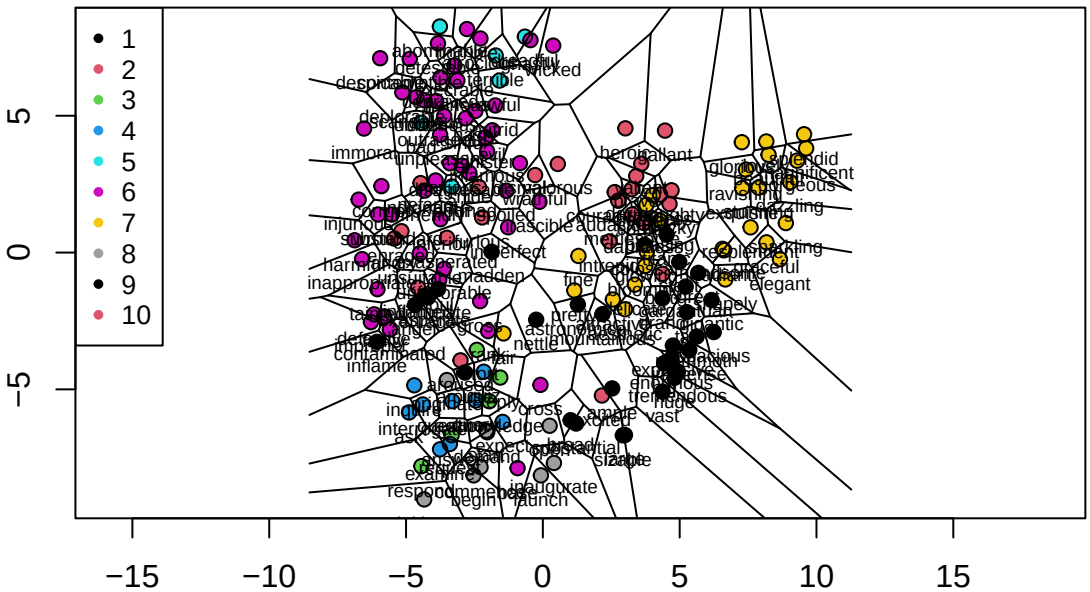
3. **Visually compare PCA, ISOMAP and t-SNE** by plotting the word2vec data, embedded into 2D using the `Plotpoints` function. Try finding the optimal k value for ISOMAP's nearest neighbour.

```r
#Principal component analysis
fitPCA <- prcomp(mydata, center = TRUE, scale. = TRUE)
```

```r
PlotPoints(fitPCA$x[,c(1,2)], mylabels, mycolors)
```

```
## Warning in (function (x0, y0, x1 = x0, y1 = y0, col = par("fg"), lty =
## par("lty"), : "wpoints" is not a graphical parameter
```

```
## Warning in plot.xy(xy.coords(x, y), type = type, ...): "wpoints" is not a
## graphical parameter
```




```r
#t-SNE (T-Distributed Stochastic Neighbor Embedding)
#install.packages('tsne')
library(tsne)
fittsne <- tsne(mydata, k = 4)
```

```
## sigma summary: Min. : 0.524057890867544 |1st Qu. : 0.711912188370555 |Median : 0.813806386509011 |Mea
```

```
## Epoch: Iteration #100 error is: 14.7320418510752
```

```
## Epoch: Iteration #200 error is: 0.35914981196095
```

```
## Epoch: Iteration #300 error is: 0.341698079117306
```

```
## Epoch: Iteration #400 error is: 0.334049738237642
```

```
## Epoch: Iteration #500 error is: 0.332189423273898
```

```
## Epoch: Iteration #600 error is: 0.331422850762638
```

```
## Epoch: Iteration #700 error is: 0.330878762967577
```

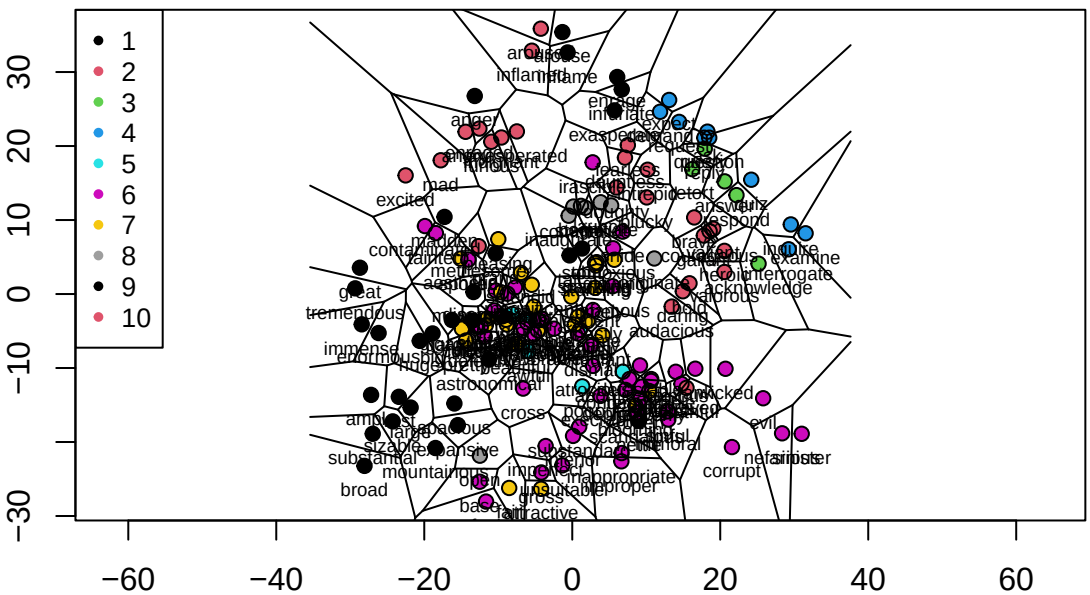
```
## Epoch: Iteration #800 error is: 0.330272458248798
```

```
## Epoch: Iteration #900 error is: 0.329927030942778
```

```
## Epoch: Iteration #1000 error is: 0.329657105341921
```

```r
PlotPoints(fittsne, mylabels, mycolors)
```

```
## Warning in (function (x0, y0, x1 = x0, y1 = y0, col = par("fg"), lty =
## par("lty"), : "wpoints" is not a graphical parameter
## Warning in (function (x0, y0, x1 = x0, y1 = y0, col = par("fg"), lty =
## par("lty"), : "wpoints" is not a graphical parameter
```



4. **Observe the effect of dimensionality reduction on a classiffication algorithm**. The supporting code in a function `Classify` performs training and testing of classification trees and gives the classification accuracy (percentage of correctly classified samples) as its result. Compare the accuracy of prediction on plain data, PCA, ISOMAP and t-SNE.

```
#classify ISOMAP
accISOMAP <- Classify(as.data.frame(fit$points), mycolors$V1, 50)

#classify PCA
accPCA <- Classify(as.data.frame(fitPCA$x), mycolors$V1, 50)

#classify t-SNE
accTSNE <- Classify(as.data.frame(fittsne), mycolors$V1, 50)

#PLOT results
print(paste("ISOMAP ACC:", mean(accISOMAP)))
```

```
## [1] "ISOMAP ACC: 0.59"
```

```
print(paste("PCA ACC:", mean(accPCA)))
```

```
## [1] "PCA ACC: 0.743333333333333"
```

```
print(paste("t-SNE ACC:", mean(accTSNE)))
```

```
## [1] "t-SNE ACC: 0.653333333333333"
```

PCA has similar accuracy as plain data. Other functions have lower.